Fetching Data with React

useState · useEffect · fetch()

Using the FakeStore API

`https://fakestoreapi.com/products/1`

Complete Code + Real API Output + Explanation

#	Topic
1	What is FakeStore API?
2	Real API Response (JSON Output)
3	Step-by-Step Code Explanation
4	Complete React Component Code
5	What the Output Looks Like (UI)
6	Handling Loading & Error States
7	Displaying Each JSON Field

1. What is FakeStore API?

FakeStore API is a **free, real REST API** used for practicing frontend development. It provides product data (like an e-commerce store) without needing your own backend. You can GET products, users, carts, and more — no API key required.

Endpoint	Description
GET /products	Get all products (20 items)
GET /products/1	Get single product by ID
GET /products?limit=5	Get first 5 products
GET /products/categories	Get all category names
GET /products/category/electronics	Get products by category

■■ *The URL we will use in this tutorial: https://fakestoreapi.com/products/1*

2. Real API Response — JSON Output

When you call **https://fakestoreapi.com/products/1**, the server returns this exact JSON object. This is the real data we received from the API:

API RESPONSE (https://fakestoreapi.com/products/1)


```
{
  "id"          : 1,
  "title"       : "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",
  "price"       : 109.95,
  "description": "Your perfect pack for everyday use and walks in the forest.",
  "category"    : "men's clothing",
  "image"       : "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
  "rating"      : {
    "rate"      : 3.9,
    "count"     : 120
  }
}
```


JSON Fields Explained:

Field	Type	Value (from real response)	Meaning
id	Number	1	Unique product ID
title	String	Fjallraven - Foldsack No. 1 Backpack...	Product name
price	Number	109.95	Price in USD
description	String	Your perfect pack for everyday use...	Product description
category	String	men's clothing	Product category
image	String	https://fakestoreapi.com/img/...	Product image URL
rating.rate	Number	3.9	Average star rating
rating.count	Number	120	Number of reviews

3. Step-by-Step Code Explanation

Step 1 — Import Hooks

Import useState and useEffect from React. These are the two hooks we need.

```
import React, { useState, useEffect } from 'react';
```

Step 2 — Declare States

We need 3 state variables: one to hold the product data, one for loading status, one for errors.

```
const [product, setProduct] = useState(null);    // stores API data
const [loading, setLoading] = useState(true);    // true while fetching
const [error,   setError]   = useState(null);    // stores error message
```

Step 3 — fetch inside useEffect

We call the API inside useEffect with an empty [] so it runs once when the component mounts.

```
useEffect(() => {
  // define async function inside
  const fetchProduct = async () => {
    try {
      const res  = await fetch('https://fakestoreapi.com/products/1');
      if (!res.ok) throw new Error('Failed to fetch!');
      const data = await res.json();    // parse JSON
      setProduct(data);                 // save to state
    } catch (err) {
      setError(err.message);            // save error
    } finally {
      setLoading(false);                // always stop loading
    }
  };
  fetchProduct();
}, []);  // [] = run once on mount
```

Step 4 — Conditional Rendering

Show different UI based on the current state — loading spinner, error message, or the actual product.

```
if (loading) return <p>Loading product...</p>;
if (error)   return <p>Error: {error}</p>;
if (!product) return null;
```

Step 5 — Display the Data

Now render each field from the product object in JSX.

```
return (
  <div>
```

```
    <h2>{product.title}</h2>
    <p>Price: ${product.price}</p>
    <p>Category: {product.category}</p>
    <p>{product.description}</p>
    <img src={product.image} alt={product.title} width='120' />
    <p>Rating: {product.rating.rate} / 5 ({product.rating.count} reviews)</p>
  </div>
);
```

4. Complete React Component Code

Here is the full, working React component that fetches product #1 from FakeStore API and displays all its fields:

```
// ProductCard.jsx  —  Complete Component
import React, { useState, useEffect } from 'react';

function ProductCard() {
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);

  useEffect(() => {
    const fetchProduct = async () => {
      try {
        const res = await fetch(
          'https://fakestoreapi.com/products/1'
        );
        if (!res.ok) {
          throw new Error(`HTTP error! Status: ${res.status}`);
        }
        const data = await res.json();
        setProduct(data);
        setError(null);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };
    fetchProduct();
  }, []);   // runs once on mount

  // ■■ Conditional rendering ■■
  if (loading) {
    return <div className='loader'>Loading product... please wait</div>;
  }
  if (error) {
    return <div className='error'>Error: {error}</div>;
  }
  if (!product) return null;

  // ■■ Render product ■■
  return (
```

```
    <div  className='product-card'>
      <img
        src={product.image}
        alt={product.title}
        style={{ width: '150px' }}
      />
      <h2>{product.title}</h2>
      <p className='price'>${product.price}</p>
      <p  className='category'>Category:  {product.category}</p>
      <p className='desc'>{product.description}</p>
      <div className='rating'>
        <span>Rating: {product.rating.rate} / 5</span>
        <span>  ({product.rating.count} reviews)</span>
      </div>
    </div>
  );
}
export default ProductCard;
```

5. What the Output Looks Like (UI)

After the component mounts and the fetch completes successfully, this is what gets rendered on screen using the real API data:

PRODUCT CARD — Rendered Output

[Product Image] 81fPKd-2AYL._AC_SL1500_.jpg

Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops

$ 109.95

Category: men's clothing

Your perfect pack for everyday use and walks in the forest. Stash your laptop (up to 15 inches) in the padded sleeve, your everyday essentials in the main compartment.

Rating: 3.9 / 5 (120 reviews)

Loading State Output:

```
Loading product... please wait
```

Error State Output:

```
Error: Failed to fetch!

(shown when network fails or API is down)
```

6. How State Changes During Fetch

The 3 state variables change as the fetch progresses:

Phase	product	loading	error	What user sees
① Initial render	null	true	null	Loading... message
② Fetch starts	null	true	null	Loading... message
③ Success	{id:1, ...}	false	null	Product card displayed
④ Error (fail)	null	false	'Failed'	Error message shown

Why finally{} Block?

The finally{} block runs whether the fetch **succeeds or fails**. This guarantees setLoading(false) is always called, so the user never stays stuck on a loading screen.

```
try {
  // fetch succeeds → setProduct(data) runs here
} catch (err) {
  // fetch fails  → setError(err.message) runs here
} finally {
  setLoading(false);  // ALWAYS runs — success or failure
}
```

■■ ***Note:*** *Without finally{}, if fetch fails, loading stays true forever and the user sees a loading spinner that never stops.*

7. Displaying Each JSON Field in JSX

Here is exactly how to access and display each field from the API response:

JSON Field	JSX Code	Real Output
id	{product.id}	1
title	{product.title}	Fjallraven - Foldsack No. 1...
price	${product.price}	$109.95
description	{product.description}	Your perfect pack...
category	{product.category}	men's clothing
image (as src)	src={product.image}	(displays the image)
rating.rate	{product.rating.rate}	3.9
rating.count	{product.rating.count}	120

Nested Object Access — rating:

The **rating** field is a nested object inside the product. Use dot notation to access nested values:

```
// The rating object looks like this:
// "rating": { "rate": 3.9, "count": 120 }

// Access nested fields with double dot notation:
product.rating.rate      // → 3.9
product.rating.count     // → 120

// In JSX:
<p>Rating: {product.rating.rate} / 5</p>
<p>Reviews: {product.rating.count}</p>
```

■ ***Tip:*** *Always check if the nested object exists before accessing it: product?.rating?.rate to avoid 'Cannot read property of undefined' errors.*

8. Quick Reference Cheat Sheet

What	Code	Notes
Import hooks	`import {useState,useEffect} from '`	Always at top
Product state	`const [product,setProduct]=useSta`	`n(u=n)` data yet
Loading state	`const [loading,setLoading]=useSta`	`tr(u=e=fe)` tching
Error state	`const [error,setError]=useState(n`	`n)` = no error
Call API once	`useEffect(()=>{...},[]);`	Empty [] = mount only
Fetch the URL	`await fetch('fakestoreapi.com/pro`	`R=tus/n's'P)` romise
Check status	`if(!res.ok) throw new Error(...)`	Doesn't auto-throw 4xx
Parse JSON	`const data = await res.json()`	Converts to JS object
Save to state	`setProduct(data)`	Triggers re-render
Show loading	`if(loading) return <p>Loading...<`	`p>B` efore main render
Show error	`if(error) return <p>{error}</p>`	Before main render
Display title	`{product.title}`	Fjallraven Backpack...
Display price	`${product.price}`	$109.95
Display image	`src={product.image}`	Use as img src attr
Nested rating	`{product.rating.rate}`	3.9 (nested object)